Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Tingan Lyu	tlyu@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea

and a paragraph describing how that paper supports your design concept.
        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider the project below.
        - Create an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the chempy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.

Students in the 21st century require new ways of distracting themselves from the daily challenges they face in the education system. Historically, this has been achieved with the use of websites like coolmathgames.com. However, these sources of entertainment and distraction have become more dated. My goal is to create a clicker tycoon game entirely playable in the browser. The game will feature many elements familiar to the clicker tycoon genre, but it will also combine new developments in culture and gaming, such as gameplay of other games on the sidebar and minigame mechanics. I aim to make this website accessible on lower-end hardware, so my hope is that the existence of this website will make the world a slightly more terrible place to live in.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

2015-2019 Era Roblox Tycoon Games - These games utilize the same basic structure I will be copying. The player starts the game by building a “dropper” and has the opportunity to upgrade their factory to produce more money. These games often have limited amounts of content. Progression is typically difficult in these games to funnel money into microtransactions.

Universal Paperclips - An incremental clicker game. The game has an elaborate story and many layers of gameplay. This game has a lot of content, but progression is slow. The game also loses its charm after sustained play.

Stimulation Clicker - An incremental clicker game. The game doesn't take very long to beat. What is notable is that it is the first game that attempts to appeal to the extremely modern and young audience. Notable innovations found in this game include "pop-out" windows playing Subway Surfers gameplay and ASMR.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

My website can be used for recreation and entertainment. Ideally, someone can play for a little bit each day to reduce their stress. The website will reward persistence, endurance, and a strong will in the face of adversity. After the player passes a certain score, they will "win" the game, and they will be able to look back on their time using this website with fondness.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The program will allow the user to accumulate a score. Ways to increase that score are dictated by the rules of the program and actions it offers to the players. The program will tell the user their current score and factors impacting that score / the rate at which it increases. The player will gain a sense of pride and accomplishment through the accumulation of the score.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The website tracks how many times the user clicks the buttons. The website may also request audio from the user's microphone. The website may ask the user to input a specific set of notes representing music. The website may ask the user for a photo to evaluate its brightness. The website may track the typing speed of the user.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The computer will increment the accumulated value by a specific amount each time the button is pressed. Eventually, this will be automatic, and the factor by which it increments the data may be modified by the quality of other inputs. Factors that impact the accumulation of data may also have their impacts evaluated by the program's code based on pre-defined criteria. E.g., a base score rate of x may be multiplied by y (rate = xy). The value of y might be determined by the user's WPM in a typing test with the formula y = log(wpm).

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will get back a value representing their accumulated score and a list of the possible actions they can take to increase the rate of accumulation of their score. There may also be pages to view and manage other mechanics that affect the accumulation rate of their score.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Score - some integer representing both a score in the traditional sense and a currency that the player can spend to improve their score accumulation rate.
As mentioned above, the player will have access to other mechanics that act as minigames. The player's performance in these minigames will have a positive impact on how fast they can accumulate score.
Some examples I have in mind are:
Loudness - in db of a input taken from a player
%color - how many% of a picture a player uploads is of a certain color
Word_length - the longest word that the user can come up with which satisfies a certain condition
Typing_speed (wpm) - the number of words the user types / time it took to type these words in minutes.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Score -
Details I need:
- *Current score*
- *Rate of accumulation from main page*
- *Rate of accumulation boost from secondary pages/minigames*
- *Costs / score spent on upgrades / secondary pages / minigames*

Details I will ignore:
- *Score is arbitrary and contained within the game. It has no relation with reality. All details related to score are details I need.*

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

- *The minigames placed on other pages that can impact the score can be difficult to implement.*
- *Finding a way to ensure that accumulation is represented in "per second" rather than some time dependent on how fast the user's hardware can complete a cycle of calculations.*
- *Creating the basic tycoon gameplay will be a repetitive process that might need to be automated with AI to complete in a reasonable amount of time.*

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

When using an LLM, I will ask it to generate comments for the code it produces. I will also ask it to explain complicated coding techniques that appear in the generation. I will test the website after each revision to make sure that it runs as intended. After every iteration, I will ask the AI to walk through the code changes part by part. I will also track variables in real time to make sure everything is working as intended. I am less worried about hallucinations

than shallow learning because an LLM could probably get the code to work eventually if it is fed with the proper documentation and existing code.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

Create a website that I will be intrinsically motivated to work on and fulfills all final project requirements.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

Prompt:
Uploaded Copy of CS1 Final Project 0_ Pick a Problem.pdf
“im working on a final project for a compsci class and I'm currently filling out part 5. For my project, what are some of the minigames that affect score accumulation I can implement that an LLM can conceivably cook up with low token use, using only Python, the Drafter library, and no other library imports?”

Response:

Based on your project proposal, I think you have a lot of room to implement interesting minigames without making the project unnecessarily complicated.

Your project is essentially a clicker tycoon with secondary minigames that modify your score-per-second multiplier. You've already identified four possible mechanics in Part 3:

- *Loudness (microphone input)*
- *Percentage of a certain color in an uploaded image*
- *Longest word satisfying a particular condition*
- *Typing speed (WPM)*

I'd recommend expanding these into a collection of small, independent minigames, each of

which produces a numerical result. That result can then be converted into a multiplier or bonus to your passive score accumulation.

One important constraint: with only Python and Drafter, you should favor minigames that use:

- *Text input*
- *Button presses*
- *Sliders or numerical input*
- *Comparisons against predefined answers*
- *Lists, strings, and basic arithmetic*
- *Simple random-number generation*

Avoid mechanics that require real-time graphics, audio processing, image processing, or JavaScript browser APIs. Those would be considerably more difficult under your restrictions.

1. Minigames that are relatively easy to implement

These are the ones I'd prioritize. They require very little code and are well within the capabilities of introductory Python.

A. Word challenge

Very easy
Text input

The player receives a condition and must submit a word satisfying it.

Possible challenges:

- *Longest word beginning with a particular letter.*
- *Longest word containing a particular sequence of letters.*
- *Word containing the most vowels.*
- *Word with exactly a specified number of letters.*
- *Word that contains no instances of a particular letter.*

Scoring: Award points based on word length, with bonuses for satisfying more difficult conditions.

Implementation: Store the player's input as a string, check its length, and use basic string operations to evaluate the conditions.

B. Mental arithmetic challenge

Very easy
Numerical input

Present the player with a mathematical expression and ask them to solve it.

Possible variations:

- *Addition and subtraction.*
- *Multiplication tables.*
- *Order of operations.*
- *Finding missing numbers in equations.*
- *Solving simple algebraic expressions.*

Scoring: Award points for correct answers, potentially with higher rewards for more difficult questions.

Implementation: Compare the player's numerical input against a predefined answer.

C. Number guessing challenge

Very easy
Numerical input

Give the player a number range and ask them to guess a specific number.

Possible variations:

- *Guess a number between 1 and 100.*
- *Guess a number based on clues such as whether it is odd or even.*
- *Guess a number based on whether it is greater or less than another number.*
- *Guess a number using a limited number of attempts.*

Scoring: Award more points for fewer attempts.

Implementation: Use numerical comparisons, conditional statements, and a counter.

D. Memory challenge

Easy
Text input

Display a sequence of characters or numbers and ask the player to reproduce it.

Possible variations:

- *Memorize a sequence of numbers.*
- *Memorize a sequence of letters.*
- *Memorize a list of words.*
- *Memorize a sequence of mathematical operations.*

Scoring: Award points based on the number of correctly reproduced characters.

Implementation: Store a predefined sequence as a string and compare it against the player's input.

E. Trivia challenge

Easy
Buttons

Present the player with multiple-choice questions.

Possible categories:

- *General knowledge.*
- *Geography.*
- *Mathematics.*
- *Science.*
- *Computer science.*
- *Aviation, if you want to incorporate a personal interest.*

Scoring: Award points for correct answers, with more difficult questions granting larger bonuses.

Implementation: Store questions and answers in lists or dictionaries and compare the selected answer against the correct answer.

F. Pattern recognition

Easy
Text/numerical input

Present the player with a sequence and ask them to identify the next element.

Examples:

Sequence	***Answer***
2, 4, 6, 8, ?	*10*
1, 2, 4, 8, ?	*16*

3, 6, 12, 24, 48
?

1, 4, 9, 16, 25
?

Scoring: Award points for correct answers, with more complex patterns granting greater rewards.

Implementation: Compare numerical input against a predefined answer.

G. Word unscrambling

Easy
Text input

Display a scrambled word and ask the player to reconstruct it.

For example:

- `RTAEHC` → `CHATER`
- `NIGOPRMMRA` → `PROGRAMMING`
- `RTEUPMOC` → `COMPUTER`

Scoring: Award points for correctly identifying the word, potentially based on word length.

Implementation: Store scrambled words and their solutions in a dictionary. Compare the player's input against the solution.

2. Slightly more interesting minigames

These require a little more thought but are still realistic for your project.

H. Resource allocation challenge

Easy–moderate
Numerical input

Give the player a limited amount of money or resources and ask them to distribute it between several investments.

For example, they have 100 credits to distribute between three factories. Each factory has a

different return on investment.

Scoring: Calculate the total return based on their allocation.

Implementation: Use numerical inputs and basic arithmetic. You could even reuse your main game's upgrade mechanics.

I. Efficiency challenge

Easy–moderate
Numerical input

Give the player a problem involving limited resources and ask them to find the most efficient solution.

For example:

- *How many products can be manufactured with a given quantity of resources?*
- *How many machines are needed to meet a production target?*
- *What is the most efficient combination of upgrades?*

Scoring: Reward players for finding solutions that maximize output while minimizing resource consumption.

J. Reaction challenge

Moderate
Button presses

Display a target number and ask the player to click a button as many times as possible before reaching it.

Alternatively, have the player press a button a specified number of times without exceeding a target.

Scoring: Award points based on how close they get to the target.

Implementation: Maintain a counter in the Drafter state and increment it whenever the button is pressed.

One limitation: a genuine reaction-time or timed clicking game would require reliable timing functionality. I'd stick to the untimed version unless your Drafter setup provides a suitable timer.

K. Mathematical estimation

Easy–moderate
Numerical input

Present the player with a calculation and ask them to estimate the answer without using exact arithmetic.

For example:

- *Estimate the square root of 150.*
- *Estimate the result of 49 × 21.*
- *Estimate the percentage increase from 80 to 120.*

Scoring: Award more points for answers closer to the correct value.

Implementation: Use subtraction, absolute values, and basic arithmetic to calculate the difference between the player's answer and the correct answer.

3. How I'd connect these minigames to your clicker game

I would avoid making each minigame directly award a massive quantity of score. Instead, have each minigame generate a performance rating that modifies your passive score accumulation.

For example:

Minigame	***Performance metric***	***Effect***
Word challenge	*Word length*	*+1% score rate per point*
Arithmetic	*Correct answers*	*+2% score rate per answer*
Memory	*Characters remembered*	*+1% score rate per character*
Trivia	*Questions answered correctly*	*+5% score rate per question*

Pattern recognition	*Correct answers*	*+3% score rate per answer*
Resource allocation	*Investment return*	*Bonus based on efficiency*

You could also make these bonuses temporary, requiring players to revisit minigames periodically, or permanent, allowing players to gradually build up their accumulation rate.

I particularly like the idea of making the minigames purchasable. For example, the player starts with only the word challenge and must spend accumulated score to unlock arithmetic, trivia, memory, and so on.

4. What I'd avoid given your restrictions

Your original ideas are interesting, but some are significantly harder than they initially appear.

Original idea	***Difficulty***	***Reason***
Loudness measurement	*High*	*Requires microphone access and audio processing*
Image color percentage	*High*	*Requires image decoding and pixel analysis*
Longest word	*Very low*	*Straightforward string operations*
Typing speed	*Moderate–high*	*Requires reliable timing and text input*

Memory challenge	*Very low*	*String comparison*
Trivia	*Very low*	*Basic conditional logic*
Arithmetic	*Very low*	*Basic mathematical operations*

One additional restriction worth remembering: Python's `random` module requires an import. If your instructor prohibits all imports besides Drafter, use predefined question sequences rather than randomly generated questions.

5. My recommendations

If I were selecting minigames for this project, I'd prioritize these five:

1. Word challenge

Best overall

Very easy to implement, lots of possible variations, and directly builds on your original idea.

2. Memory challenge

Simple implementation with an interesting gameplay loop.

3. Resource allocation

Integrates particularly well with the tycoon genre and introduces meaningful decision-making.

4. Trivia

Easy to expand with additional questions and difficulty levels.

5. Pattern recognition

Offers a straightforward way to introduce progressively more difficult challenges.

One architectural suggestion: Make every minigame return a numerical performance value. Have a separate function convert that value into a score-per-second bonus. This way, all your minigames interact with the same accumulation system, and adding new minigames

won't require rewriting your core game logic.

For Part 5 of your worksheet, the most valuable takeaway is that you can implement several distinct minigames using nothing more than Drafter components, Python strings, lists, dictionaries, conditionals, and arithmetic. You don't need complicated browser interactions to achieve the variety you're looking for.

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The AI suggested simple minigames like unscrambling words or finding the longest word with a certain condition. It also warned about making minigames that require image or audio processing or require the use of random (which would mean extra Python imports). I have no idea if extra imports besides what Python comes with and drafter are allowed; I remember reading they weren't a few weeks ago, but I can't find the line on that in the requirements. Overall, the AI suggested a way simpler website than what I had in mind. Most of the minigames it suggested seemed to be reasonably implementable. I also did place an extra focus on the minigames that affect score accumulation rather than the tycoon gameplay.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

Prompt 1:
"okay for part 5 bullet 3 in the pdf explain how J. Reaction challenge is so"

Response 1:

For Part 5, Bullet 3 (The Pushback), you need to identify the most complicated feature suggested, explain why it's too ambitious for an introductory programming project, and ask for a simpler alternative.

I'd use the Reaction Challenge (J) as your example. Here's a draft you can adapt:

I think the reaction challenge might be too complicated for this project. Ideally, I would like the player to click a button as quickly as possible within a set time limit, with their performance affecting their score accumulation rate. However, implementing a reliable timer and measuring the player's reaction speed using only Python and Drafter could be difficult.

It may also require functionality outside the scope of the libraries I am permitted to use.

Could you suggest a simpler alternative that retains the general concept of a reaction-based minigame but does not require precise timing or additional library imports?

Why this works:

- *Identifies a specific feature that could be technically challenging.*
- *Explains the limitations imposed by your library restrictions.*
- *Establishes what functionality you want to preserve.*
- *Gives the LLM an opportunity to suggest a simpler implementation.*

It also sets you up nicely for Bullet 4, where you can explain how your project became more focused by replacing complicated real-time mechanics with simpler, turn-based minigames.

Prompt 2:

Use the draft to answer the question

Response 2:

A simpler alternative would be a Target Click Challenge.

Instead of measuring how quickly the player reacts, have them repeatedly click a button to reach a randomly selected target number without exceeding it.

For example:

- *The player is given a target of 25.*
- *Each click increases their counter by 1.*
- *The player can choose when to stop.*
- *Reaching exactly 25 awards the maximum bonus.*
- *Finishing below 25 awards a smaller bonus.*
- *Exceeding 25 results in no bonus.*

Why this is easier to implement:

- *No timer is necessary.*
- *No precise measurements are required.*
- *The player's progress can be tracked using a simple integer variable.*
- *Button presses can be handled using Drafter's built-in functionality.*
- *The scoring system only requires basic conditional statements.*

Additional possibilities:

- *Introduce different target numbers with different rewards.*
- *Allow certain upgrades to increase the reward.*

- *Give players a limited number of attempts.*
- *Introduce buttons that increase the counter by different amounts.*

This retains the original idea of a reaction-based minigame because players must decide when to click and when to stop, but replaces the technically challenging timing component with a much simpler decision-making mechanic.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

I think the website will be more text-focused than what I originally planned for. I had expected audio- or visual-based minigames to play a greater role in what affects player score accumulation. Unless I have 20 million tokens for Claude and a will to see to it that some Texan town only gets brown AI Datacenter water to drink for the next 50 years, my website might be only slightly above the basic requirements of the project. This is all written without me actually having any experience with vibecoding. It could be that LLMs are better than I expected, in which case I will expand the project scope if I have time. Using the suggestions provided by the AI as a bare minimum for the minigames to implement. **The conversations in part 5 also do not address the main tycoon gameplay.** This is because I believe the tycoon will actually be the easiest part to implement. The tycoon gameplay will just rely on a score tracked by State and the revealing of various buttons to increase the score directly or increase the rate at which the score increases. On the other hand, minigames are more difficult, but they are needed to satisfy the project requirements.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

LLM, Google search, Stack Overflow (search and read only, no posting). Reading the Drafter library might be helpful for implementing inputs for the minigames,

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

1. *Ask myself whether I need that code, if not, press Ctrl+Z and move on.*
2. *Screenshot the error and ask the AI (if I have remaining conversations / tokens), ask*

it to reference Drafter library documentation specifically.

3. *Ask AI to make a debugging tool that tracks State at all times (remember to remove before deploying).*
4. *Go through the website with said debugging tool.*
5. *If out of tokens or can't talk to the AI for whatever reason, read the error and work through the code manually.*
6. *Reference online resources and drafter library and identify the error type and its meaning.*
7. *Write assert_equals for the new function and run the programe without start_server()*
8. *If possible, start deleting half of the new code at a time to see what is causing the issue.*
9. *Cry, get sleep, hope things are better tomorrow.*

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Prompt 1:
look at the documentation for the drafter python library and generate a list of all the possibilities for a website made in drafter over the span of 1 week, assume ~2-4 hours of work on the website per week

Response 1:

Drafter website project ideas: What can you build in one week?

For a CS1 student using Drafter, with about 2–4 hours of work per week, I'd focus on websites that solve one clear problem, take some user input, process it with Python, and display a useful result.

Your uploaded project worksheet emphasizes exactly this approach: define a problem, identify inputs and outputs, model the important data, and keep the scope manageable.

CS1 Final Project Alpha F26.pdf

Drafter is designed for introductory Python web development. Its strengths include building interactive pages, managing application state, working with lists and dictionaries, and presenting results without requiring students to build a separate backend. Its published research also describes plotting, image manipulation, testing, and GitHub Pages deployment.

arXiv

I'm interpreting your time limit as one week total, with 2–4 hours of development — not a multiweek project. Below is a broad catalog of realistic possibilities, from straightforward projects to ambitious ones that might fit if you already understand the relevant features.

1. What Drafter can do

The official documentation lists a much wider set of possibilities than basic text-and-button websites. These are the main building blocks you can combine.

Drafter Documentation+1
StudyBuddy | Devpost

Interactive websites

Text boxes, buttons, checkboxes, dropdowns, date and time inputs, multiple pages, forms, and navigation.

Examples: planners, quizzes, calculators, recommendation tools.

Habit Tracker Dashboard — UX/UI Concept by Anzhelika on Dribbble

Data and calculations

Python functions, loops, conditions, lists, dictionaries, dataclasses, sorting, filtering, summaries, and scores.

Examples: grade calculators, trackers, budget planners, data analyzers.

Python:Matplotlib | pyplot | .subplot() | Codecademy

Visualizations

Tables, progress bars, meters, Matplotlib graphs, images, SVG graphics, and custom layouts.

Examples: statistics dashboards, workout logs, survey analyzers.

How to make a store locator map | PamPam Blog

Browser features

Maps, geolocation, timers, repeating clocks, camera input, audio playback, microphone input, and audio recording.

Examples: location-based guides, reaction games, sound experiments, countdowns.

Plotting Lanchain REPL Code in Streamlit - LLMs and AI - Streamlit

Files and data tools

File uploads, downloads, text-file analysis, CSV processing, and supported Python libraries such as NumPy and pandas.

Examples: word counters, spreadsheet analyzers, flashcard importers.

The documentation covers these features, including their components and data types.

Drafter Documentation+1

2. The full idea catalog

Here are 60 possible website ideas, organized by category. The difficulty ratings are my estimates for a CS1 student working 2–4 hours in one week, not official Drafter ratings.

- *Easy*
  One primary feature, straightforward calculations, minimal state.
- *Moderate*
  Multiple routes, more complicated logic, or saved data.
- *Stretch*
  A specialized component, external data, or several features that must work together.

A. School and studying

1. Grade calculator

Enter assignment scores and weights; calculate a current or projected grade.

Easy

2. Study planner

Enter subjects and tasks; generate a simple study schedule.

Easy

3. Flashcard quiz

Display questions, reveal answers, and track correct responses.

Easy

4. Multiple-choice quiz maker

Choose answers, calculate a score, and show feedback.

Easy

5. Assignment tracker

Add assignments, due dates, completion status, and priorities.

Moderate

6. Study-time analyzer

Enter time spent studying and graph time by subject.

Moderate

7. GPA estimator

Enter course grades and credits; calculate an estimated GPA.

Easy

8. Citation organizer

Enter source details and produce consistently formatted references.

Moderate

9. Vocabulary trainer

Practice definitions, track correct answers, and repeat missed words.

Moderate

10. Pomodoro study timer

Use a countdown for study sessions and breaks.

Moderate

B. Productivity and personal organization

11. Habit tracker

Check off daily habits and calculate completion percentages.

Easy

12. Daily task dashboard

Display a prioritized list of tasks and completed items.

Easy

13. Goal progress tracker

Set targets and calculate progress toward each goal.

Easy

14. Countdown website

Calculate time until an exam, trip, birthday, or deadline.

Easy

15. Random task picker

Select a task from a list when you cannot decide what to do.

Easy

16. Decision helper

Score choices against user-defined criteria and rank them.

Moderate

17. Journal prompt generator

Generate prompts and let users write entries during the session.

Easy

18. Personal dashboard

Combine a clock, tasks, goals, and a small progress chart.

Moderate

19. Weekly time budget

Allocate hours across activities and compare totals with available time.

Easy

20. Reading tracker

Record books, pages, reading goals, and progress.

Moderate

C. Money, shopping, and everyday calculations

21. Budget calculator

Enter income and expenses; calculate the remaining balance.

Easy

22. Savings goal calculator

Calculate how much to save per week to reach a target.

Easy

23. Bill splitter

Divide a shared bill among people, including optional tip.

Easy

24. Discount calculator

Compare original and discounted prices.

Easy

25. Subscription cost analyzer

Calculate monthly and yearly costs from recurring expenses.

Easy

26. Grocery list organizer

Add items, quantities, and estimated prices; total the cost.

Moderate

27. Unit converter

Convert temperatures, distances, weights, or other simple units.

Easy

28. Travel budget planner

Estimate costs for transport, lodging, food, and activities.

Moderate

29. Recipe scaler

Adjust ingredient quantities based on serving count.

Easy

30. Meal-planning organizer

Choose meals from a small built-in list and create a shopping list.

Moderate

D. Games, entertainment, and creativity

31. Choose-your-own-adventure

Use buttons to navigate a branching story with multiple endings.

Moderate

32. Personality quiz

Score answers and return a result based on simple rules.

Easy

33. Trivia challenge

Ask questions, keep score, and display the final result.

Easy

34. Number guessing game

Generate a number and give higher-or-lower feedback.

Easy

35. Rock-paper-scissors

Let a player choose and calculate the winner.

Easy

36. Word scramble

Shuffle letters and check the user's guess against an answer.

Moderate

37. Random story generator

Combine user inputs and word lists into a funny story.

Easy

38. Drawing canvas

Let users draw with the browser canvas, if comfortable with its API.

Stretch

39. Color palette generator

Generate colors and display matching color combinations.

Moderate

40. Soundboard

Play preset sounds or simple generated tones using audio components.

Moderate

E. Health, fitness, and lifestyle organization

These ideas focus on general tracking and planning, not medical diagnosis or personalized treatment.

41. Workout log

Record activities, duration, and personal notes.

Easy

42. Hydration reminder planner

Set a personal daily goal and track entries during the session.

Easy

43. Sleep schedule calculator

Enter sleep and wake times to calculate the interval between them.

Easy

44. Walking or running log

Record distances and times; calculate average pace.

Moderate

45. Mood journal

Choose a mood label and write a note, then summarize entries.

Moderate

46. Stretch-break timer

Run a countdown and display a different reminder after each interval.

Moderate

47. Personal routine builder

Arrange morning, evening, or other routine steps in a checklist.

Easy

48. Screen-time budget planner

Allocate a self-chosen time budget across activities.

Easy

F. Data, maps, and useful utilities

49. CSV data analyzer

Upload a compatible CSV and calculate totals, averages, or category counts.

Stretch

50. Word-frequency analyzer

Upload or enter text and count how often each word appears.

Moderate

51. Personal statistics dashboard

Enter a small dataset and display a graph with summary statistics.

Moderate

52. Survey results visualizer

Enter answers and show counts or percentages by response.

Moderate

53. Map pinboard

Display a map with user-selected markers or a few preset places.

Stretch

54. Nearby-place marker

Request browser location and display it on a map if permission is granted.

Stretch

55. Photo filter experiment

Upload an image and apply a simple supported image transformation.

Stretch

56. Audio visualizer

Use microphone input to display sound-level changes.

Stretch

57. Typing speed tester

Measure a typing exercise and calculate speed and accuracy.

Moderate

58. Reaction-time game

Show a visual cue and measure a user's response interval.

Stretch

59. Personal file converter

Transform supported text or CSV data into a downloadable format.

Stretch

60. Random workout or activity picker

Select an activity from a user-defined list and track completed choices.

Easy

These 60 ideas are a brainstorming catalog, not a promise that every feature can be completed in four hours. The official documentation includes examples for maps, timers, audio, files, camera input, and plotting that can help you evaluate the more specialized options.

Drafter Documentation+1

3. The best ideas if you only have 2–4 hours

If the goal is to finish a working website in one week, I would prioritize ideas that can be built around one main Python function and a small amount of state.

Grade Calculator
Safest choice

1. Grade calculator

Estimated time: 2–3 hours

- *Input: assignment grades and weights.*
- *Processing: calculate a weighted average.*

- *Output: current grade and projected outcomes.*

Why: Clear input-process-output model, easy to test, and useful to students.

Medium
Best for fun

2. Trivia quiz

Estimated time: 2–4 hours

- *Input: selected answers.*
- *Processing: compare answers and calculate the score.*
- *Output: score, feedback, and optionally a results summary.*

Why: Easy to demonstrate and provides several opportunities to use conditionals and lists.

Dashboard Design for Students by Talie Abbasi on Dribbble
Best practical tool

3. Study planner

Estimated time: 3–4 hours

- *Input: subject, task, and estimated duration.*
- *Processing: organize tasks and calculate total study time.*
- *Output: a readable task list and summary.*

Why: Demonstrates state, lists, forms, and useful application design.

Word Frequency Table - Browser Text Metrics Tool
Best for data processing

4. Word-frequency analyzer

Estimated time: 3–4 hours

- *Input: a paragraph of text.*
- *Processing: split text into words and count occurrences.*
- *Output: total words, most common words, and a frequency table.*

Why: Strong demonstration of strings, dictionaries, loops, and data abstraction.

Savings Goal Tracker with Progress Bar for One Big Purchase | Koder.ai
Best simple calculator

5. Savings goal calculator

Estimated time: 2–3 hours

- *Input: target amount, current savings, and target date.*
- *Processing: calculate the remaining amount and required saving rate.*
- *Output: a weekly target and progress summary.*

Why: Simple mathematics with a clear, polished result.

These are planning estimates, not guarantees. Time spent learning Drafter, debugging, and styling can change them.

4. How to decide what is feasible

The main constraint is not just the number of components. It is how much logic and integration each feature requires.

Feature	***One-week feasibility***	***Main consideration***
Forms, buttons, navigation	*Very high*	*Core Drafter skills*
Calculations and quizzes	*Very high*	*Mostly basic Python*
Lists and progress tracking	*High*	*Requires managing state*
Tables and charts	*High*	*Charts need suitable data*
Downloading a generated file	*High*	*Keep the format simple*
Uploading and analyzing CSV files	*Medium*	*File parsing and invalid inputs*

Browser timers and audio	*Medium*	*Component behavior and testing*
Maps and geolocation	*Medium–low*	*Permissions and network access*
Camera and image manipulation	*Medium–low*	*Additional data types and image processing*
External APIs	*Variable*	*Network access, API limits, and response formats*

The official documentation explains that Drafter applications run in the browser without a conventional backend or database. State is generally kept in browser memory and can disappear when the page is closed or reloaded. Published applications should not be assumed to have shared accounts or centralized storage.

Drafter+1

This means you should avoid making the core of a one-week project depend on:

- *User accounts and secure login.*
- *Multiple users sharing live data.*
- *A custom database or server.*
- *Complex live collaboration.*
- *A complicated external API integration.*
- *Several advanced features that all depend on one another.*

Local browser storage or downloadable files may help with persistence, but you should verify the exact approach supported by your course's Drafter environment.

5. Build a project plan around your available time

For a four-hour week, a sensible schedule is:

30 minutes

Define the problem

Write down the user, the inputs, the processing, and the output.

60 minutes

Build the core function

Get the main calculation or algorithm working with test data.

60 minutes

Connect the Drafter interface

Add the inputs, button, state, and results page.

45 minutes

Test and improve

Try normal, empty, and unusual inputs; fix errors and improve the layout.

25 minutes

Prepare the final demo

Test the complete flow and make sure the project can be run or published as required.

Total: 4 hours. If you have only 2 hours, reduce the number of features rather than skipping testing.

6. Narrow down the options

You can use this quick selector to identify ideas that fit your interests and time.

What type of website interests you most?

School and studying
Productivity and organization
Games and entertainment
Data analysis and visualization
Calculators and everyday tools
Creative projects, maps, or audio

How much time do you have?

Prompt 2:
I made a typo, consider the same prompt but 2-4 hours of work per day over a week

Response 2:

That changes the possibilities quite a bit. With 2–4 hours per day for 7 days, you have roughly 14–28 hours total. That's enough for a CS1 student to build a more complete Drafter website with multiple pages, persistent local data, charts, or one carefully chosen advanced feature.

I'd now consider projects that have a polished interface, a meaningful core algorithm, and several supporting features—not just a single calculator.

What becomes realistic with 14–28 hours?

Project scope	***What you could build***
Small	*A polished quiz, calculator, or interactive game*
Medium	*A multi-page planner with saved data, filtering, and summaries*
Ambitious	*A data dashboard with graphs, file uploads, or maps*
Stretch	*A specialized audio, image, or API-based application, if the integration works in your environment*

The biggest improvement is that you have time to build, test, debug, and polish the application rather than stopping as soon as the main feature works.

7. Expanded catalog: projects worth considering with a full week

Here are the projects I'd especially consider now that you have more time. The scope estimates include development and testing, assuming you know basic Python but are still learning Drafter.

Dashboard Design for Students by Talie Abbasi on Dribbble

1. Student command center

14–20 hours

Combine an assignment tracker, study timer, weekly schedule, completion statistics, and a dashboard.

Possible standout feature: Automatically prioritize assignments based on due date and estimated work time.

Moneta - Money Management Dashboard by Elux UI/UX for Elux Space on Dribbble

2. Personal budget analyzer

12–18 hours

Enter expenses, organize them into categories, calculate totals, display charts, and export a report.

Possible standout feature: Compare spending categories against user-defined budgets.

Brain Bricks | AI-Powered Quiz Learning From Any Content

3. Custom quiz and flashcard platform

12–20 hours

Create questions, practice flashcards, track scores, revisit missed answers, and save question sets locally.

Possible standout feature: A review mode that prioritizes questions answered incorrectly.

Free CSV Analyzer Tool — Explore CSV Files Online

4. Data exploration studio

16–24 hours

Upload a CSV, select columns, calculate statistics, filter rows, and generate graphs using pandas and Matplotlib if supported.

Possible standout feature: Let the user choose which variables appear on a graph.

Schools - Storyhaven

5. Branching story game

12–18 hours

Build a story with multiple paths, inventory items, player choices, branching endings, and

saved progress.

Possible standout feature: Choices change a player's resources or unlock future scenes.

Google Maps Saved Places: The Essential Traveler's Guide

6. Local exploration guide

18–28 hours

Show preset places on a map, filter by category, save favorites locally, and provide details about each location.

Possible standout feature: A personalized list of places based on selected interests.

Keep a preset dataset as a fallback in case maps or external requests cause problems.

Habit Tracker Dashboard — UX/UI Concept by Anzhelika on Dribbble

7. Habit and goal tracker

12–18 hours

Create habits, log daily completions, review trends, set goals, and store entries in local browser storage.

Possible standout feature: Weekly summaries and a graph of progress over time.

[App Fridays] Stashcook, a free recipe management and meal planner service, is a delight | YourStory

8. Recipe and meal planner

14–22 hours

Browse preset recipes, scale ingredient quantities, select meals, and generate a consolidated shopping list.

Possible standout feature: Combine ingredients from multiple recipes and sum duplicate items.

These estimates are approximate planning judgments, not documented benchmarks. Map integration, CSV parsing, browser storage, and other specialized features should be tested early rather than left until the end.

8. A realistic seven-day schedule

For a medium-sized project, here's how I would divide 21 hours of work.

Day 1

2–3 hours

Research and design

Pick the problem, sketch the pages, define your data structures, and confirm the necessary Drafter components.

Day 2

2–4 hours

Build the core logic

Write and test the Python functions that solve the main problem.

Day 3

2–4 hours

Build the main interface

Create the page layouts, inputs, buttons, and navigation.

Day 4

2–4 hours

Add data and state

Implement adding, editing, filtering, calculations, or local persistence.

Day 5

2–4 hours

Build one standout feature

Add a graph, timer, export function, map, or another feature that improves the core experience.

Day 6

2–4 hours

Test and polish

Fix bugs, handle invalid inputs, improve styling, and test the complete workflow.

Day 7

2–3 hours

Final checks

Test the deployed application, verify data behavior, and prepare a demonstration.

9. My recommendation

For a CS1 final project, I would choose a project with these four elements:

1. *A clear purpose that solves a real problem.*
2. *Meaningful Python logic, such as sorting, filtering, scoring, or analyzing data.*
3. *An interactive interface with multiple inputs and useful outputs.*
4. *One impressive extra feature, such as a graph, local saving, or a downloadable report.*

A student command center, quiz platform, or data exploration studio could meet all four goals. The best choice depends on what you find interesting and which Drafter features you are already comfortable using.

One important constraint: Drafter can run entirely in the browser and supports static deployment, but it does not automatically provide a server or shared database. Plan around local data and self-contained functionality unless your course environment explicitly supports something else.

arXiv

Prompt 3:
okay how many of the things are actually possible in the drafter library and how many of the things need extra imports?

Response 3:

1. What the official Drafter library already includes

I checked the current official documentation, especially its complete component reference and "What Drafter can do" guide. Drafter includes built-in components for forms, tables, progress bars, graphs, maps, timers, audio, file uploads, downloads, and more.

Drafter Documentation+1

The key distinction is that a feature can be built into Drafter even when the Python code you write needs to use other libraries or browser APIs.

For example, Drafter has a built-in `Map` component, but calculating the best route between multiple places is a separate problem.

2. Breakdown of the 60 ideas

Here's my estimate for the original list of 60 website ideas.

42

Drafter + basic Python

No additional library imports needed beyond Drafter and the usual dataclass setup.

12

Standard-library imports may help

Modules such as `random`, `datetime`, `time`, `csv`, or `json`. These come with Python and aren't external packages.

6

Additional library or browser API work

A plotting library, image processing, or custom browser interaction may be needed for the full proposed feature.

These are approximate classifications of the 60 ideas, based on their proposed versions. Some ideas can move between categories depending on implementation.

3. Which features need what?

Feature	Drafter already	Other imports or work

	provides	
Grade calculator	*Forms, buttons, output*	*None beyond basic Python*
Quiz or flashcards	*Buttons, forms, state*	`random` *if questions are randomized*
Study planner	*Forms, lists, tables*	*None for a basic version*
Habit tracker	*State, progress bars, tables*	*Optional date handling or local-storage integration*
Countdown or Pomodoro timer	`Timer`, `Clock`	*Usually none for basic timer behavior*
Budget dashboard	*Forms, tables*	*Matplotlib for graphs*
CSV analyzer	`FileUpload`, `Download`, *tables*	`csv` *or optionally* `pandas`
Data visualization	`MatPlotLibPlot`	`matplotlib`, *potentially* `pandas` *or* `numpy`
Map explorer	`Map`, `CurrentLocation`	*Optional external data or mapping services*
Soundboard	`Audio`, `Tone`, `Sound`	*Usually none for built-in playback*

Audio visualizer	`Microphone`	*Potential browser API work for custom visualization*
Photo filter	`Camera`*, image components*	*Image-processing library or browser API*
Drawing application	`Canvas`	*Custom browser interaction for drawing behavior*

The built-in components are documented in the [Drafter component reference]. The official guide also explains how to use other Python libraries and access browser features.

Drafter Documentation+1

An important distinction

An extra import doesn't necessarily mean extra installation.

- `random`*,* `datetime`*,* `time`*,* `csv`*, and* `json` *are part of Python's standard library.*
- `matplotlib`*,* `pandas`*, and* `numpy` *are separate packages.*
- *Browser APIs may require JavaScript interoperability, even when no third-party Python package is necessary.*

Drafter's documentation says that it can install compatible PyPI libraries for browser execution, but not every Python library works in that environment.

Drafter Documentation

4. What I'd recommend for your project

With your 14–28 total hours over one week, I wouldn't avoid additional imports entirely. I'd choose a project whose main functionality works with Drafter and basic Python, then add at most one extra technical feature.

Three particularly good options are:

- *Study planner: multiple pages, assignment tracking, prioritization, and progress bars.*
- *Quiz platform: randomized questions, scoring, review of missed answers, and progress tracking.*

- *CSV data analyzer: file uploads, statistics, a table, and one Matplotlib graph.*

The first two can be built without external packages. The third is more technically ambitious but gives you a chance to use an additional library in a meaningful way.

One final caveat: the counts above are estimates rather than a formal audit of every implementation. Drafter supplies the interface components, but you still need to write the Python logic that makes each website useful.

Prompt 4:
can you make draggable elements that slot into boxes with drafter and only python imports?

Response 4:

Yes, but with an important limitation: Drafter can display boxes and elements, but true drag-and-drop behavior generally requires custom browser-side JavaScript or HTML event handling. Drafter's documentation supports adding JavaScript to a page and interacting with browser elements through JavaScript interoperability.

Drafter Documentation+1

What you can build

Example: Drag items into matching boxes

Interactive mockup: select an item, then choose a box. This demonstrates the sorting logic, not actual mouse dragging.

Items

AppleCarrotPear
FruitClick to place selected item
VegetablesClick to place selected item
Check answersReset

Can you do it with only Python imports?

- *Yes: Build the boxes, represent their contents with Python lists, and use buttons to move items between boxes.*
- *Partially: You can create the visual layout with Drafter components and custom HTML/CSS.*
- *Not with standard Drafter components alone: Genuine mouse-based dragging and dropping, where an element follows the cursor and snaps into a target box, needs*

browser-side event handling.

Drafter itself provides a documented route for adding JavaScript and custom HTML, so you wouldn't necessarily need a third-party library. However, that would mean writing some JavaScript, even if all your application logic remains in Python.

Drafter Documentation+1

My recommendation: If this is for your one-week project, genuine drag-and-drop is feasible as a stretch feature. If your assignment requires Python-only code, use clickable items and destination boxes instead; you can still build a polished sorting game, matching exercise, or visual organizer.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.